Multi-Company Management Portal

Finalized Tech Stack

This is the finalized tech stack for the portal. Every piece below is chosen for the long term, meaning it is meant to carry the product from launch through scale to thousands or millions of users without being swapped out later. There is no separate testing stack or early stage stack, this is the one we build on and the one we scale on.

Frontend and Backend

Next.js (TypeScript)	One codebase and one language for both the pages the owner and their team see and the server logic behind them. Each request runs in isolation, so one failing request cannot degrade the experience for every other user on the portal.

Document Processing

Separate Node.js worker service	Document uploads and extraction take real time and cannot run inside a normal web request. This runs as its own long running service, kept apart from the main app so a slow or heavy job never affects anyone browsing the portal at the same time.
BullMQ with Redis (job queue)	Manages the extraction jobs and any other background task. Multiple workers can run in parallel during high volume, and failed jobs retry automatically instead of silently failing and leaving a company's data incomplete.
Multimodal AI model	Reads the uploaded document directly and returns structured dates and obligations, cross-checked against a jurisdiction rules table we maintain ourselves. This always runs through the queue, never while a user is waiting on a page. We are not using plain OCR or hardcoded rules for this part, because every company's document is worded differently, there is no fixed pattern to write rules for. The AI reads and understands the document itself, not just the text on the page.

Since this is only for UAE companies, here is exactly where OCR is enough and where we will need AI. OCR (a small Python system, it reads the text off the page and pulls out labeled dates using simple rules) works fine for documents from authorities we already know, DED and the major free zones like DIFC, ADGM, JAFZA, DMCC, RAKEZ, as long as the format stays consistent and the date is clearly labeled. The moment a document comes from an authority we have not built rules for yet, or the scan is unclear, or the date is written as a sentence instead of a clean label, that is where OCR falls short and we need AI, since it actually understands the document instead of just reading text off it.

Database, Auth, and Storage

Supabase (Postgres), Pro plan with dedicated compute	Gives every company its own isolated access through Row Level Security, enforced at the database level rather than relying on every API route to filter correctly. This is what actually keeps one company's data invisible to another, regardless of plan tier. We move to the Team plan once we are onboarding enterprise-scale clients who require formal compliance guarantees.

Hosting (Open Question, Need Your Input)

We have two options for where our app and worker will run. Both keep everything on one platform, which is what we want. Sharing both below, would like your thoughts before we lock this in.

Option 1: Railway	Costs more for the same amount of power. If something crashes, it restarts on its own within seconds, no setup needed from us. Handling more traffic is a setting we change, not a server we build. Less ongoing work for us.
Option 2: Hostinger VPS	Costs less for the same amount of power. We get full control over the server, but if something crashes, nothing restarts it unless we build that ourselves. Also, if we ever need to reboot the server, for a security update or any other reason, the portal will be down for a few minutes during that reboot, since there is no second server picking up traffic in the meantime. To be as reliable as Option 1, we would need at least two servers running together with a load balancer between them, plus someone keeping the server patched and monitored on an ongoing basis.

My take: leaning towards Railway since we do not have a dedicated person for server maintenance right now, but wanted everyone's opinion before we finalize this piece.

Caching

Redis (shared with the queue)	Caches permission and feature access checks so they are not queried from the database on every page load. Uses infrastructure already in the stack rather than adding a new piece.

Security

Row Level Security + server-side only keys	Every table holding company data is scoped by company ID with default-deny policies. The Supabase service role key never reaches the browser. This is the actual guarantee against data leaking between companies or to the open internet, independent of which plan we are on.

Estimated Monthly Cost (For the First Few Hundred Companies)

This is what running the whole stack costs each month at our starting size, a few hundred companies. The cost will grow later as more owners join, this number is just for getting started.

Hosting (Railway)	Runs the app, the worker, and Redis together. About 20 to 40 dollars a month.
Database (Supabase Pro)	The plan itself is a fixed 25 dollars a month. A small extra amount gets added once we use more storage or compute than what is included, usually landing around 25 to 35 dollars total.
Document reading (AI model)	About 20 to 100 dollars a month, depending on how many documents get uploaded.

Total for this starting size: roughly 70 to 150 dollars a month. I will share updated numbers once we are closer to thousands of companies, since the cost grows in steps as we upgrade plans.